Bài 1.11 – Chuẩn hóa dữ liệu, chia train/validation/test

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Mô phỏng phỏng vấn*

🎯 Sau bài này bạn sẽ

- Hiểu vì sao các cột có đơn vị khác nhau làm mô hình "hiểu sai", và sửa bằng **chuẩn hóa**.
- Tính tay và dùng Scikit-learn cho **Min-Max** và **Standardization**.
- Chia dữ liệu thành **train / validation / test** đúng cách: xáo trộn, stratify, đặt seed.
- Nhận ra và tránh **rò rỉ dữ liệu (data leakage)**, lỗi khiến mô hình "đẹp trên giấy, hỏng ngoài đời".

---

1. Mô phỏng phỏng vấn 🎤

Bạn đang phỏng vấn vị trí Junior ML Engineer. Hãy **tự trả lời thành tiếng** trước khi đọc gợi ý.

> **Người phỏng vấn:** "Em có dataset khách hàng với 2 cột: tuổi (18–70) và thu nhập (5–200 triệu/tháng). Em dùng thuật toán tính khoảng cách (như KNN) để tìm khách hàng giống nhau. Có vấn đề gì không?"

> **Người phỏng vấn:** "Em chia dữ liệu 80% train, 20% test. Trước khi chia, em chuẩn hóa toàn bộ dữ liệu cho tiện. Cách làm đó sai ở đâu?"

> **Người phỏng vấn:** "Mô hình của em đạt độ chính xác 98% trên tập test. Em có tin con số đó không? Em sẽ kiểm tra những gì?"

Ba câu này là câu hỏi phỏng vấn **thật** và rất phổ biến. Cuối bài (mục 7) có gợi ý trả lời. Sau bài học, bạn sẽ trả lời được cả ba.

---

2. Vấn đề: đơn vị khác nhau làm sai lệch mọi thứ

2.1. Ví dụ với khoảng cách

Ba căn hộ: [diện tích (m^2), số phòng ngủ]:

- A = [60, 2]
- B = [61, 5] (gần bằng diện tích A, nhưng **nhiều hơn 3 phòng**)
- C = [75, 2] (cùng số phòng, rộng hơn **15 m^2**)

Căn nào giống A hơn? Với người mua, chênh 3 phòng ngủ là khác biệt **rất lớn** (căn hộ 2 phòng ngủ khác hẳn căn 5 phòng ngủ). Chênh 15 m^2 thì vừa phải.

```
import numpy as np

A = np.array([60, 2]); B = np.array([61, 5]); C = np.array([75, 2])
print(np.linalg.norm(A - B).round(2), np.linalg.norm(A - C).round(2))   # khoảng cách Euclid (bài 1.8)
```

Kết quả mẫu:

```
3.16 15.0
```

Máy kết luận **B giống A hơn nhiều** (3,16 so với 15). Vì sao? Diện tích tính bằng chục m^2, số phòng chỉ là 1–5. Cột có **con số lớn hơn** "lấn át" hoàn toàn cột kia, bất kể cột nào quan trọng hơn.

2.2. Ví dụ với Gradient Descent

Nhớ lại bài 1.9: x có giá trị 4–8 (không nằm quanh 0) làm **b học cực chậm** (2.000 epoch vẫn chưa tới). Và khi đổi x sang m^2 (40–80), learning rate phải giảm 100 lần.

Chuẩn hóa đưa mọi cột về cùng "thang đo", giải quyết cả hai vấn đề.

---

3. Hai cách chuẩn hóa

3.1. Min-Max Normalization (Chuẩn hóa min-max)

- **Định nghĩa:** Co giãn dữ liệu về khoảng **[0, 1]**: nhỏ nhất thành 0, lớn nhất thành 1.
- **Ví dụ đời thực:** Đổi điểm thi từ các thang khác nhau (thang 10, thang 100) về phần trăm.

Công thức:

$$x' = \frac{x - min}{max - min}$$

Ký hiệu	Ý nghĩa
x	Giá trị gốc
min, max	Nhỏ nhất, lớn nhất của cột
x'	Giá trị sau chuẩn hóa, nằm trong [0, 1]

Tính thử: diện tích [40, 50, 60, 70, 80]. min = 40, max = 80.

- 40 → (40 − 40) / 40 = **0**
- 50 → (50 − 40) / 40 = **0,25**
- 80 → (80 − 40) / 40 = **1**

3.2. Standardization (Chuẩn hóa chuẩn tắc / z-score)

- **Định nghĩa:** Biến đổi để cột có **trung bình = 0** và **độ lệch chuẩn = 1**. Mỗi giá trị cho biết "cách trung bình bao nhiêu độ lệch chuẩn".
- **Ví dụ đời thực:** "Chiều cao của bạn cao hơn trung bình 1,5 độ lệch chuẩn" (bài 1.7, z-score).

Công thức:

$$z = \frac{x - \mu}{\sigma}$$

Ký hiệu	Ý nghĩa
μ (đọc "muy")	Trung bình của cột
σ (đọc "xích-ma")	Độ lệch chuẩn của cột
z	Giá trị sau chuẩn hóa

Tính thử: diện tích [40, 50, 60, 70, 80]. μ = 60. σ = √[(400 + 100 + 0 + 100 + 400) / 5] = √200 ≈ 14,142. (Scikit-learn chia cho n, giống NumPy, bài 1.7.)

- 40 → (40 − 60) / 14,142 ≈ **−1,414**
- 60 → **0**
- 70 → 10 / 14,142 ≈ **0,707**

```
dien_tich = np.array([40, 50, 60, 70, 80], dtype=float)

mm = (dien_tich - dien_tich.min()) / (dien_tich.max() - dien_tich.min())   # min-
max
print(mm)

z = (dien_tich - dien_tich.mean()) / dien_tich.std()                       #
standardization
print(dien_tich.mean(), dien_tich.std().round(3), z.round(3))
```

Kết quả mẫu:

```
[0.   0.25 0.5  0.75 1.  ]
60.0 14.142 [-1.414 -0.707  0.     0.707  1.414]
```

3.3. Dùng Scikit-learn

Scikit-learn là thư viện ML cổ điển phổ biến nhất của Python (sẽ dùng xuyên suốt Giai đoạn 2). Các công cụ chuẩn hóa của nó theo đúng khuôn `fit` / `transform` bạn đã tự viết ở bài 1.2.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler

X = pd.DataFrame({"dien_tich": [40, 50, 60, 70, 80], "phong_ngu": [1, 2, 2, 3,
3]})

print(MinMaxScaler().fit_transform(X).round(3))   # fit + transform trong 1 lệnh
```

```
sc = StandardScaler()
Xs = sc.fit_transform(X)            # fit: HỌC μ và σ của từng cột; transform: ÁP
công thức
print(Xs.round(3))
print(sc.mean_, sc.scale_.round(3)) # μ và σ đã học được
```

Kết quả mẫu:

```
[[0.   0.  ]
 [0.25 0.5 ]
 [0.5  0.5 ]
 [0.75 1.  ]
 [1.   1.  ]]
[[-1.414 -1.604]
 [-0.707 -0.267]
 [ 0.    -0.267]
 [ 0.707  1.069]
 [ 1.414  1.069]]
[60.   2.2] [14.142  0.748]
```

Sau chuẩn hóa, hai cột cùng thang đo, không cột nào lấn át cột nào.

3.4. Kiểm chứng: khoảng cách sau chuẩn hóa

```
# 5 căn hộ: A, B, C và 2 căn khác để scaler học được μ, σ hợp lý
Xk = np.array([[60, 2], [61, 5], [75, 2], [40, 1], [80, 4]], dtype=float)
Xk_s = StandardScaler().fit_transform(Xk)
print(np.linalg.norm(Xk_s[0] - Xk_s[1]).round(2),    # A–B
      np.linalg.norm(Xk_s[0] - Xk_s[2]).round(2))    # A–C
```

Kết quả mẫu:

```
2.04 1.07
```

Giờ **C mới là căn giống A hơn** (1,07 < 2,04). Đúng với trực giác.

3.5. Kiểm chứng: Gradient Descent sau chuẩn hóa

Chạy lại mô hình ở bài 1.9 với x đã chuẩn hóa, learning rate 0,1 (gấp 10 lần lần trước):

```
x = np.array([4, 5, 6, 7, 8], dtype=float)
y = np.array([2.0, 2.6, 2.9, 3.6, 4.0])
xs = (x - x.mean()) / x.std()               # chuẩn hóa x
```

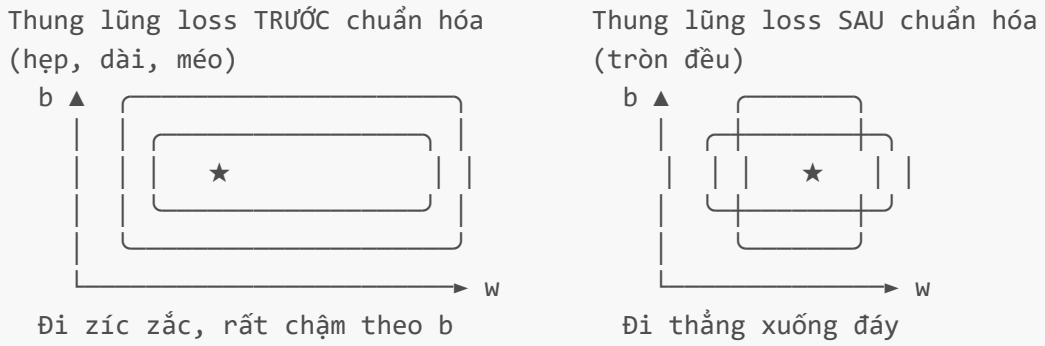
```
w, b, lr = 0.0, 0.0, 0.1
for epoch in range(101):
    sai_so = w * xs + b - y
    loss = (sai_so ** 2).mean()
    w -= lr * 2 * (sai_so * xs).mean()      # -= là viết tắt của w = w - ...
    b -= lr * 2 * sai_so.mean()
    if epoch in [0, 10, 50, 100]:
        print(f"Epoch {epoch:>3}: loss = {loss:.5f} | w = {w:.4f} | b = {b:.4f}")

# Đổi w, b về đơn vị gốc để so sánh với đáp án chính xác (w = 0.5, b = 0.02)
print("w gốc =", round(w / x.std(), 4), "| b gốc =", round(b - w * x.mean() /
x.std(), 4))
```

Kết quả mẫu:

```
Epoch   0: loss = 9.62600 | w = 0.1414 | b = 0.6040
Epoch  10: loss = 0.11652 | w = 0.6464 | b = 2.7606
Epoch  50: loss = 0.00560 | w = 0.7071 | b = 3.0200
Epoch 100: loss = 0.00560 | w = 0.7071 | b = 3.0200
w gốc = 0.5 | b gốc = 0.02
```

Chỉ **50 epoch** đã tới **đúng** đáp án (w = 0,5; b = 0,02). Trước khi chuẩn hóa, 2.000 epoch vẫn chưa tới. Để ý b sau chuẩn hóa = 3,02 = **giá trung bình**: vì x đã nằm quanh 0, b đơn giản là "giá khi diện tích ở mức trung bình".



3.6. Chọn cách nào?

	Min-Max	**Standardization**
Kết quả	Nằm trong [0, 1]	Trung bình 0, ĐLC 1, không giới hạn
Với outlier	**Rất kém**: một outlier ép mọi giá trị khác dồn vào một góc	Khá hơn (nhưng vẫn bị ảnh hưởng)
Hay dùng cho	Ảnh (độ sáng pixel 0–255 → 0–1), mạng nơ-ron	**Mặc định cho hầu hết mô hình** (Linear/Logistic Regression, KNN, SVM)

Không cần chuẩn hóa: các mô hình dạng cây (Decision Tree, Random Forest, XGBoost) vì chúng chỉ so sánh "lớn hơn / nhỏ hơn" trong từng cột (sẽ học kỹ ở bài 2.9).

---

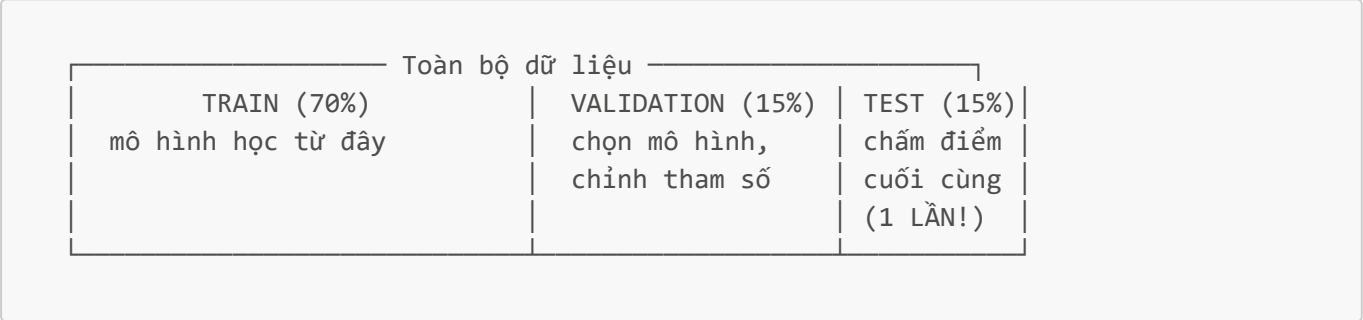
4. Chia dữ liệu: train / validation / test

4.1. Vì sao phải chia?

So sánh ví von: Học sinh ôn thi.

- **Train set (Tập huấn luyện)** = sách bài tập có lời giải. Học sinh học từ đây.
- **Validation set (Tập kiểm định)** = các đề thi thử. Dùng để **điều chỉnh cách học** (học thêm phần nào, đổi phương pháp).
- **Test set (Tập kiểm tra)** = đề thi thật. Chỉ làm **một lần duy nhất** ở cuối, để biết năng lực thật.

Nếu chấm điểm học sinh bằng chính đề trong sách bài tập, học sinh **học thuộc lòng** đáp án sẽ đạt 10 điểm, nhưng gặp đề mới thì trượt. Mô hình ML cũng vậy: nó có thể "học thuộc" dữ liệu huấn luyện. Hiện tượng này gọi là **overfitting (quá khớp)** (sẽ học kỹ ở bài 2.6).



Tỉ lệ phổ biến: 70/15/15 hoặc 80/10/10. Dữ liệu càng lớn (hàng triệu dòng), phần test càng có thể nhỏ (1–2% vẫn đủ nhiều).

4.2. Cái bẫy: dữ liệu đã được sắp xếp

Dataset California được **sắp xếp theo kinh độ** (từ đông sang tây):

```
from sklearn.model_selection import train_test_split

url = "https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv"
df = pd.read_csv(url)
df.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
              "dan_so", "so_ho", "thu_nhap", "gia_nha"]
print(df["kinh_do"].head(3).tolist(), df["kinh_do"].tail(3).tolist())
```

Kết quả mẫu:

```
[-114.31, -114.47, -114.56] [-124.3, -124.3, -124.35]
```

Nếu cắt 80% đầu làm train, 20% cuối làm test **mà không xáo trộn**:

```
cat = int(len(df) * 0.8)                          # vị trí cắt
train_sai, test_sai = df.iloc[:cat], df.iloc[cat:]
print("Không xáo trộn → kinh độ TB train:", train_sai["kinh_do"].mean().round(2),
      "| test:", test_sai["kinh_do"].mean().round(2))
print("                 giá TB train:", round(train_sai["gia_nha"].mean()),
      "| test:", round(test_sai["gia_nha"].mean()))

# Cách đúng: train_test_split tự xáo trộn trước khi chia
train, test = train_test_split(df, test_size=0.2, random_state=42)   #
random_state = seed
print("Có xáo trộn     → kinh độ TB train:", train["kinh_do"].mean().round(2),
      "| test:", test["kinh_do"].mean().round(2))
print("                 giá TB train:", round(train["gia_nha"].mean()),
      "| test:", round(test["gia_nha"].mean()))
print(train.shape, test.shape)
```

Kết quả mẫu:

```
Không xáo trộn → kinh độ TB train: -118.85 | test: -122.39
                 giá TB train: 197507 | test: 246476
Có xáo trộn     → kinh độ TB train: -119.56 | test: -119.58
                 giá TB train: 206708 | test: 209673
(13600, 9) (3400, 9)
```

Không xáo trộn: tập test toàn là **vùng ven biển phía tây** (gồm San Francisco), giá trung bình cao hơn train gần 50.000 USD. Mô hình học ở vùng này rồi bị thi ở vùng khác, **không công bằng**. Có xáo trộn: train và test giống nhau về phân bố.

> ⚠️ **Ngoại lệ quan trọng:** dữ liệu **theo thời gian** (doanh thu theo ngày, giá cổ phiếu) thì **không được xáo trộn**. Phải lấy quá khứ làm train, tương lai làm test, vì ngoài đời bạn luôn dự đoán tương lai từ quá khứ.

4.3. Chia 3 phần

`train_test_split` chỉ chia làm 2, nên gọi 2 lần:

```
X = df.drop(columns="gia_nha")      # đặc trưng (input)
y = df["gia_nha"]                   # cột đích (output cần dự đoán)

# Lần 1: tách 15% làm test
X_tam, X_test, y_tam, y_test = train_test_split(X, y, test_size=0.15,
random_state=42)
# Lần 2: từ 85% còn lại, tách ra phần bằng 15% tổng làm validation
X_train, X_val, y_train, y_val = train_test_split(X_tam, y_tam, test_size=0.15 /
0.85, random_state=42)
```

```
print(X_train.shape, X_val.shape, X_test.shape)
print(round(len(X_train) / len(X), 3), round(len(X_val) / len(X), 3),
round(len(X_test) / len(X), 3))
```

Kết quả mẫu:

```
(11900, 8) (2550, 8) (2550, 8)
0.7 0.15 0.15
```

Quy ước đặt tên: chữ **X hoa** cho bảng đặc trưng (ma trận, bài 1.8), chữ **y thường** cho cột đích (vector).

4.4. Stratify: giữ đúng tỉ lệ các lớp

Bài toán phát hiện gian lận: 1.000 giao dịch, chỉ **5%** là gian lận. Chia 10% làm test (100 giao dịch). Nếu xui, tập test có thể chỉ có 3 vụ gian lận, hoặc 6 vụ, thay vì đúng 5.

```
nhan = np.array([1] * 50 + [0] * 950)          # 50 gian lận (1), 950 bình thường
(0)
Xd = np.arange(1000).reshape(-1, 1)            # đặc trưng giả, chỉ để minh họa
for strat in [None, nhan]:
    ti_le = []
    for seed in range(5):                      # thử chia 5 lần với 5 seed khác
nhau
        _, _, _, yt = train_test_split(Xd, nhan, test_size=0.1, random_state=seed,
stratify=strat)
        ti_le.append(round(float(yt.mean()), 3))   # tỉ lệ gian lận trong tập test
    print("có stratify" if strat is not None else "không stratify", ti_le)
```

Kết quả mẫu:

```
không stratify [0.05, 0.06, 0.03, 0.04, 0.03]
có stratify [0.05, 0.05, 0.05, 0.05, 0.05]
```

Quy tắc: bài toán **phân loại** → luôn thêm `stratify=y`.

---

5. Data Leakage (Rò rỉ dữ liệu)

5.1. Khái niệm

- **Định nghĩa:** Thông tin từ tập test (hoặc từ tương lai) **lọt vào** quá trình huấn luyện, khiến mô hình đạt điểm cao giả tạo.
- **Ví dụ đời thực:** Học sinh lén xem đề thi thật trước khi thi. Điểm cao, nhưng không phản ánh năng lực.

5.2. Lỗi phổ biến nhất: chuẩn hóa trước khi chia

```
❌ SAI                                  ✅ ĐÚNG
1. scaler.fit(TOÀN BỘ dữ liệu)          1. Chia train / val / test
   → μ, σ có "nhìn thấy" tập test       2. scaler.fit(CHỈ train)
2. Chia train / test                    3. scaler.transform(train)
                                        4. scaler.transform(val),
transform(test)
                                           (dùng μ, σ của train)
```

```python
from sklearn.preprocessing import StandardScaler

sc_sai = StandardScaler().fit(X)          # ❌ học μ từ cả tập test
sc_dung = StandardScaler().fit(X_train)   # ✅ chỉ học từ train
print(sc_sai.mean_[7].round(4), sc_dung.mean_[7].round(4))   # cột 7 = thu_nhap

X_train_s = sc_dung.transform(X_train)    # áp μ, σ của train lên train
X_test_s = sc_dung.transform(X_test)      # áp μ, σ của TRAIN lên test (không fit lại!)
print(X_train_s[:, 7].mean().round(3), X_train_s[:, 7].std().round(3))
print(X_test_s[:, 7].mean().round(3), X_test_s[:, 7].std().round(3))
```

Kết quả mẫu:

```
3.8836 3.8689
-0.0 1.0
0.045 1.005
```

(`-0.0` chính là 0.) Train có trung bình 0, ĐLC 1 chính xác. Test thì **gần** 0 và 1, **không chính xác tuyệt đối**. Đó là **điều đúng**: tập test đại diện cho dữ liệu mới ngoài đời, mà dữ liệu mới thì không bao giờ khớp hoàn toàn với train.

Vì sao chênh lệch nhỏ vậy mà vẫn quan trọng? Ở đây chỉ lệch 0,015 vì dữ liệu lớn. Nhưng nguyên tắc sai thì lỗi sẽ lớn dần với các bước phức tạp hơn: điền giá trị thiếu, chọn đặc trưng, encoding (bài 1.6)... **Mọi thứ "học" từ dữ liệu đều chỉ được học từ train.**

5.3. Quy tắc vàng

> **Test set là "két sắt".** Chỉ mở **một lần**, ở **cuối cùng**. Mọi quyết định (chọn mô hình, chỉnh tham số, chọn đặc trưng) đều dựa trên validation.

Bài 2.17 sẽ dạy **Pipeline**, công cụ của Scikit-learn giúp tự động làm đúng quy tắc này, không sợ quên.

---

6. Bài tập

1. Lương 5 nhân viên (triệu): `[8, 10, 12, 15, 50]`. Tính tay min-max cho giá trị 15. Sau đó chạy cả min-max và standardization bằng code. Bạn thấy vấn đề gì với min-max khi có người lương 50?

2. Thử `RobustScaler` của Scikit-learn trên cùng dữ liệu. Nó dùng **trung vị** và **IQR** (bài 1.6–1.7) thay cho trung bình và ĐLC. Kết quả khác gì?
3. Một bạn viết code như dưới đây. Tìm **2 lỗi**:

```
X_s = StandardScaler().fit_transform(X)
X_train, X_test = X_s[:8000], X_s[8000:]
```

4. Bài toán dự đoán doanh thu ngày mai của cửa hàng từ dữ liệu 3 năm. Có nên dùng `train_test_split` mặc định không? Nên chia thế nào?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
luong = np.array([8, 10, 12, 15, 50], dtype=float)
print(((luong - luong.min()) / (luong.max() - luong.min())).round(3))
# [0.    0.048 0.095 0.167 1.   ]
print(((luong - luong.mean()) / luong.std()).round(3))
# [-0.702 -0.574 -0.447 -0.255  1.978]

from sklearn.preprocessing import RobustScaler
print(RobustScaler().fit_transform(luong.reshape(-1, 1)).ravel().round(3))   #
ravel: dàn phẳng về 1 chiều
# [-0.8 -0.4  0.   0.6  7.6]
```

1. Tay: (15 − 8) / (50 − 8) = 7 / 42 ≈ **0,167**. Vấn đề: chỉ một người lương 50 làm 4 người còn lại bị **ép dồn vào khoảng 0–0,17**, gần như không phân biệt được với nhau. Min-max rất nhạy với outlier.
2. RobustScaler: 4 người bình thường được trải đều từ −0,8 đến 0,6, còn outlier bị đẩy ra xa (7,6) một cách rõ ràng. Phù hợp khi dữ liệu có outlier mà bạn không muốn xóa.
3. Lỗi 1: **chuẩn hóa trước khi chia** → rò rỉ dữ liệu. Lỗi 2: **cắt theo thứ tự, không xáo trộn** → nếu dữ liệu được sắp xếp (như California), train và test khác phân bố.
4. **Không.** Dữ liệu thời gian không được xáo trộn. Nên lấy ví dụ 2,5 năm đầu làm train, vài tháng tiếp theo làm validation, vài tháng cuối làm test. Nếu xáo trộn, mô hình sẽ được "nhìn thấy tương lai" khi học (một dạng rò rỉ dữ liệu).

---

7. Gợi ý trả lời phỏng vấn (mục 1)

Câu 1 (KNN, tuổi và thu nhập): "Thu nhập có thang lớn hơn nhiều so với tuổi, nên khoảng cách gần như chỉ phụ thuộc vào thu nhập. Em sẽ chuẩn hóa (StandardScaler) cả hai cột trước khi dùng KNN. Nếu thu nhập có nhiều outlier, em cân nhắc log transform hoặc RobustScaler."

Câu 2 (chuẩn hóa trước khi chia): "Đó là data leakage: trung bình và độ lệch chuẩn được tính có cả dữ liệu test, nên kết quả đánh giá lạc quan hơn thực tế. Đúng là phải chia trước, fit scaler chỉ trên train, rồi transform cho val và test. Trong thực tế em dùng Pipeline để tránh quên."

Câu 3 (98% chính xác): "Em chưa tin ngay. Em sẽ kiểm tra:

1. Dữ liệu có mất cân bằng không? Nếu 98% mẫu thuộc một lớp, mô hình đoán bừa lớp đó cũng được 98%. (Sẽ học các chỉ số tốt hơn ở bài 2.5.)
2. Có rò rỉ dữ liệu không? Ví dụ chuẩn hóa trước khi chia, có dòng trùng lặp giữa train và test, hay có cột 'tiết lộ' đáp án.
3. Chia có đúng không? Đã xáo trộn chưa, có stratify chưa, dữ liệu thời gian có bị xáo trộn không."

---

8. Flashcard

Mặt trước	**Mặt sau**
Vì sao cần chuẩn hóa?	Để cột có số lớn không lấn át cột khác; giúp Gradient Descent hội tụ nhanh.
Công thức min-max?	(x − min) / (max − min) → [0, 1]
Công thức standardization?	(x − μ) / σ → trung bình 0, ĐLC 1
Mô hình nào không cần chuẩn hóa?	Mô hình dạng cây (Decision Tree, Random Forest, XGBoost).
Train / Validation / Test ví như?	Sách bài tập / thi thử / thi thật.
Vì sao phải xáo trộn trước khi chia?	Dữ liệu có thể đã được sắp xếp, làm train và test khác phân bố.
Khi nào KHÔNG xáo trộn?	Dữ liệu theo thời gian.
`stratify=y` để làm gì?	Giữ đúng tỉ lệ các lớp trong mỗi tập.
Data leakage là gì?	Thông tin từ test/tương lai lọt vào huấn luyện → điểm cao giả.
Fit scaler trên tập nào?	CHỈ trên train. Rồi transform cho val và test.

Bài tiếp theo: 1.12 – Mini project EDA trên dataset thật.